

# ECHO — Flex Message Card Integration

SUPERIOR PDR | Technical Spec for Myke | 2026-05-29

## Overview

When ECHO's reply contains a trigger code, the server should:

- 1. Strip the trigger code from the text (customers never see it)
- 2. Send the plain text reply first using `reply_message`
- 3. Push the corresponding Flex Message card using `push_message`

## Trigger Codes → Flex Card Files

| Trigger Code                  | JSON File                | When to Fire                      |
|-------------------------------|--------------------------|-----------------------------------|
| [SHOW_SERVICE_MENU]           | 01_service_menu.json     | After ECHO gets the customer name |
| [SHOW_SERVICE_OPTIONS_PDR]    | 03_service_no_paint.json | Photo analysis — paint intact     |
| [SHOW_SERVICE_OPTIONS_PAINT]  | 04_service_paint.json    | Photo analysis — paint damaged    |
| [SHOW_SERVICE_OPTIONS BUMPER] | 05_service_bumper.json   | Photo analysis — bumper damage    |

Drop the entire `echo_flex_messages/` folder into the ECHO server root.

## Suggested Handler (Node.js)

```
const fs = require('fs');
const path = require('path');

function loadFlex(filename) {
  return JSON.parse(fs.readFileSync(
    path.join(__dirname, 'echo_flex_messages', filename), 'utf-8'
  ));
}

const FLEX_MAP = {
  '[SHOW_SERVICE_MENU]': { file: '01_service_menu.json', altText: 'Please select a service' },
  '[SHOW_SERVICE_OPTIONS_PDR]': { file: '03_service_no_paint.json', altText: 'Please select a service option' },
  '[SHOW_SERVICE_OPTIONS_PAINT]': { file: '04_service_paint.json', altText: 'Please select a service option' },
  '[SHOW_SERVICE_OPTIONS BUMPER]': { file: '05_service_bumper.json', altText: 'Please select a service option' },
};

async function handleEchoReply(userId, replyToken, responseText, lineClient) {
  let cleanText = responseText;
  let triggerKey = null;

  for (const key of Object.keys(FLEX_MAP)) {
    if (cleanText.includes(key)) {
      cleanText = cleanText.replace(key, '').trim();
      triggerKey = key;
      break; // max one card per reply
    }
  }

  await lineClient.replyMessage(replyToken, { type: 'text', text: cleanText });

  if (triggerKey) {
    const { file, altText } = FLEX_MAP[triggerKey];
    await lineClient.pushMessage(userId, {
```

```

        type: 'flex', altText,
        contents: loadFlex(file),
    });
}
}

```

## Postback Handling

When a customer taps a button, LINE fires a postback event. Handle these data values:

| data value                                   | Action                                                 |
|----------------------------------------------|--------------------------------------------------------|
| action=dent_assessment                       | Continue vehicle assessment flow                       |
| action=branch_info                           | Reply with branch info, skip assessment                |
| action=complaint                             | Reply: hotline 0800-889-365, fire [SEND_TO_JAY]        |
| action=select_service&service=PDR            | Update CRM case: service = PDR                         |
| action=select_service&service=bankin-rough   | Update CRM case: service = rough panel repair          |
| action=select_service&service=bankin-repaint | Update CRM case: service = traditional panel + repaint |
| action=select_service&service=pdr-repaint    | Update CRM case: service = high-quality PDR + repaint  |
| action=select_service&service=PDR-bumper     | Update CRM case: service = PDR bumper (to assess)      |
| action=select_service&service=bumper-repair  | Update CRM case: service = bumper repair/replacement   |

## Rules to Keep in Mind

- [SHOW\_SERVICE\_MENU] fires only once per conversation — use a flag to prevent repeats.
- [SHOW\_SERVICE\_OPTIONS\_\*] fires only once per photo analysis round.
- Always use push\_message for the card — the reply\_message token is single-use.
- Full JSON files are in the repo under echo\_flex\_messages/ and ECHO\_Myke\_■■■■■■.docx